

06 — RNNs & Sequence Models

Time: ~4-5 hours | **Level:** Intermediate-Advanced

What you'll learn:

- Why sequences need special architectures
- Recurrent Neural Networks (RNNs): the idea of memory
- The vanishing gradient problem
- LSTM (Long Short-Term Memory): solving the memory problem
- GRU (Gated Recurrent Unit): a simpler alternative
- Sequence-to-Sequence (Seq2Seq): the precursor to Transformers
- Why RNNs lost to Transformers (and why understanding them still matters)

Prerequisites: Notebooks 03-05 (neural networks, PyTorch, NLP basics)

```
In [ ]: import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt
from torch.utils.data import DataLoader, TensorDataset

torch.manual_seed(42)
np.random.seed(42)
```

1. The Problem with Feedforward Networks for Sequences

In Notebook 05, we averaged token embeddings. This threw away **order information**:

```
"Patient was anxious THEN became calm" (improving)
"Patient was calm THEN became anxious" (worsening)
```

Both sentences have identical BoW/TF-IDF representations. We need a model that reads tokens **one at a time** and builds up a running understanding — like how you read a sentence.

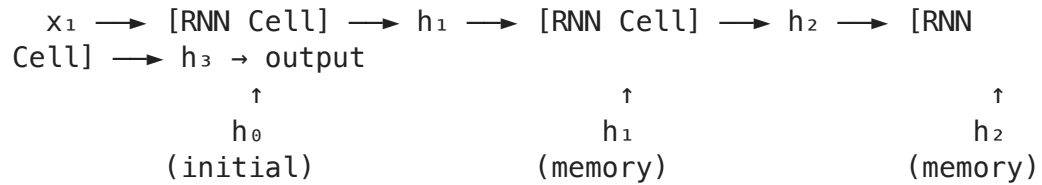
2. Recurrent Neural Networks (RNNs)

An RNN processes a sequence step-by-step, maintaining a **hidden state** h_t that serves as its memory:

$$h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b)$$

At each step t :

1. Take current input x_t (e.g., the embedding of word t)
2. Combine with previous hidden state h_{t-1} (memory from past words)
3. Apply tanh activation $\rightarrow$ new hidden state h_t



```
In [ ]: # — RNN Cell from Scratch —————

class RNNCellFromScratch:
    """
    A single RNN cell, implemented from scratch.
    This is what happens INSIDE nn.RNN at each time step.
    """
    def __init__(self, input_size, hidden_size):
        # Xavier initialisation
        scale_xh = np.sqrt(1 / input_size)
        scale_hh = np.sqrt(1 / hidden_size)
        self.W_xh = np.random.randn(input_size, hidden_size) * scale_xh
        self.W_hh = np.random.randn(hidden_size, hidden_size) * scale_hh
        self.b = np.zeros(hidden_size)
        self.hidden_size = hidden_size

    def forward(self, x_t, h_prev):
        """One step: combine input with memory → new memory."""
        z = x_t @ self.W_xh + h_prev @ self.W_hh + self.b
        h_t = np.tanh(z)
        return h_t

    def process_sequence(self, X):
        """
        Process an entire sequence.
        X: (seq_len, input_size)
        Returns: all hidden states (seq_len, hidden_size)
        """
        seq_len = X.shape[0]
        h = np.zeros(self.hidden_size) # initial hidden state
        hidden_states = []

        for t in range(seq_len):
            h = self.forward(X[t], h)
            hidden_states.append(h.copy())

        return np.array(hidden_states)

# Demo: process a sequence of 5 "tokens" (random embeddings)
rnn_cell = RNNCellFromScratch(input_size=8, hidden_size=16)
```

```

fake_sequence = np.random.randn(5, 8) # 5 tokens, 8-dim each

hidden_states = rnn_cell.process_sequence(fake_sequence)
print(f'Input sequence: {fake_sequence.shape}') # (5, 8)
print(f'Hidden states: {hidden_states.shape}') # (5, 16)
print(f'Final hidden state ("sentence summary"): {hidden_states[-1][:5].round(2)}

print(f'\n💡 The final hidden state h5 has "seen" all 5 tokens.')
print('    It\'s the RNN\'s summary of the entire sequence.')
print('    We can use it for classification, generation, etc.')

```

3. The Vanishing Gradient Problem

RNNs have a critical flaw: during backpropagation through time, gradients get **multiplied many times** by the same weight matrix. If values are < 1 , gradients shrink exponentially:

$$\frac{\partial h_{100}}{\partial h_1} = \prod_{t=2}^{100} \frac{\partial h_t}{\partial h_{t-1}} \approx 0$$

Result: The model "forgets" early tokens in long sequences. Token 1 has almost zero influence on the output.

```

In [ ]: # — Visualise vanishing gradients —————

def simulate_gradient_flow(seq_len, weight_scale=0.9):
    """Simulate how gradients shrink through an RNN."""
    gradient_norms = [1.0] # start at 1
    for t in range(seq_len - 1):
        # Each step multiplies gradient by ~weight_scale (from tanh derivative)
        gradient_norms.append(gradient_norms[-1] * weight_scale * 0.85) # t
    return gradient_norms

fig, ax = plt.subplots(figsize=(10, 5))
for scale, color, label in [
    (0.9, 'red', 'RNN (vanishing)'),
    (1.0, 'orange', 'RNN (borderline)'),
    (1.1, 'blue', 'RNN (exploding)'),
]:
    grads = simulate_gradient_flow(100, scale)
    ax.plot(grads, color=color, label=label, linewidth=1.5)

ax.set_xlabel('Time steps back (from output)')
ax.set_ylabel('Gradient magnitude')
ax.set_title('Vanishing/Exploding Gradient Problem in RNNs')
ax.set_yscale('log')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print('💡 After ~50 steps back, gradients are essentially ZERO.')

```

```
print('    The model cannot learn long-range dependencies.')
```

```
print('    This is why vanilla RNNs fail on long documents and conversations.'
```

4. LSTM — Long Short-Term Memory

LSTMs (1997) solve vanishing gradients with a **gating mechanism**:

Instead of one hidden state, LSTM maintains two:

- h_t — hidden state (short-term memory)
- c_t — cell state (long-term memory highway)

Three gates control information flow:

Gate	Symbol	Purpose
Forget	f_t	What to throw away from old memory
Input	i_t	What new info to store
Output	o_t	What to expose as hidden state

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (\text{forget gate})$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (\text{input gate})$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c) \quad (\text{candidate memory})$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (\text{new cell state})$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (\text{output gate})$$

$$h_t = o_t \odot \tanh(c_t) \quad (\text{new hidden state})$$

Key insight: The cell state c_t acts as a highway — gradients can flow through it without being multiplied by weight matrices.

```
In [ ]: # — LSTM for text classification —————
import re
from collections import Counter

# Reuse the mental health dataset from Notebook 05
templates = {
    'depression': [
        'feeling sad hopeless and tired all day',
        'lost interest in activities social withdrawal',
        'persistent sadness and low energy levels',
        'difficulty sleeping and loss of appetite',
        'feeling worthless and unable to concentrate',
    ],
    'anxiety': [
        'panic attacks with rapid heartbeat sweating',
        'constant worry about everything feeling restless',
    ],
}
```

```

        'fear of social situations avoiding crowds',
        'racing thoughts and difficulty relaxing',
        'physical tension headaches and nervousness',
    ],
}

# Generate data
train_texts, train_labels = [], []
for _ in range(100):
    for label, temps in templates.items():
        text = np.random.choice(temps)
        words = text.split()
        if np.random.random() > 0.5:
            words.append(np.random.choice(words))
        np.random.shuffle(words)
        train_texts.append(' '.join(words))
        train_labels.append(0 if label == 'depression' else 1)

# Tokenise and index
def simple_tokenise(text):
    return re.findall(r'[a-z]+', text.lower())

all_words = [w for t in train_texts for w in simple_tokenise(t)]
vocab = sorted(set(all_words))
w2i = {w: i+1 for i, w in enumerate(vocab)}
w2i['<PAD>'] = 0
vocab_size = len(w2i)

max_len = 12
def encode(text):
    tokens = simple_tokenise(text)
    ids = [w2i.get(t, 0) for t in tokens[:max_len]]
    ids += [0] * (max_len - len(ids))
    return ids

X = torch.tensor([encode(t) for t in train_texts])
y = torch.tensor(train_labels, dtype=torch.float32).unsqueeze(1)
print(f'Data: {X.shape}, Labels: {y.shape}, Vocab: {vocab_size}')

```

```

In [ ]: # — LSTM Classifier —————

class LSTMClassifier(nn.Module):
    """
    Embedding → LSTM → take last hidden state → classify.

    This was the go-to architecture for NLP before Transformers (2017).
    Google Translate used LSTM seq2seq until ~2019.
    """
    def __init__(self, vocab_size, embed_dim, hidden_dim, n_classes=1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.lstm = nn.LSTM(
            input_size=embed_dim,
            hidden_size=hidden_dim,
            num_layers=2,          # stack 2 LSTM layers
            batch_first=True,     # input shape: (batch, seq, features)

```

```

        dropout=0.3,
    )
    self.classifier = nn.Sequential(
        nn.Linear(hidden_dim, n_classes),
        nn.Sigmoid()
    )

    def forward(self, x):
        # x: (batch, seq_len) - token indices
        embedded = self.embedding(x) # (batch, seq_len, embed_dim)

        # LSTM returns: output (all hidden states), (h_n, c_n)
        output, (h_n, c_n) = self.lstm(embedded)
        # h_n shape: (num_layers, batch, hidden_dim)

        # Take the last layer's final hidden state
        last_hidden = h_n[-1] # (batch, hidden_dim)

        return self.classifier(last_hidden) # (batch, 1)

model = LSTMClassifier(vocab_size, embed_dim=32, hidden_dim=64)
print(model)
print(f'\nTotal parameters: {sum(p.numel() for p in model.parameters()):,}')

```

```

In [ ]: # — Train the LSTM —————
optimizer = optim.Adam(model.parameters(), lr=0.005)
loss_fn = nn.BCELoss()
loader = DataLoader(TensorDataset(X, y), batch_size=32, shuffle=True)

losses = []
for epoch in range(50):
    total_loss = 0
    for bx, by in loader:
        pred = model(bx)
        loss = loss_fn(pred, by)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    avg = total_loss / len(loader)
    losses.append(avg)
    if epoch % 10 == 0:
        with torch.no_grad():
            all_pred = model(X)
            acc = ((all_pred > 0.5).float() == y).float().mean()
            print(f'Epoch {epoch:2d} | Loss: {avg:.4f} | Acc: {acc:.3f}')

plt.figure(figsize=(8, 4))
plt.plot(losses, color='steelblue', linewidth=1.5)
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('LSTM Training')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```

5. GRU — Gated Recurrent Unit

GRU (2014) is a simplified LSTM with only **two gates** instead of three:

- **Reset gate** r_t : how much past to forget
- **Update gate** z_t : how much to update

Swap nn.LSTM for nn.GRU – that's literally it

```
self.gru = nn.GRU(embed_dim, hidden_dim, batch_first=True)
```

GRU has fewer parameters than LSTM and often performs comparably. Use GRU when:

- You want faster training
- Your sequences are not extremely long
- You're experimenting and want a quick baseline

6. Seq2Seq — The Bridge to Transformers

Sequence-to-Sequence (2014) maps one sequence to another:

"How are you feeling today?" → [Encoder LSTM → Context vector
→ Decoder LSTM] → "I'm feeling anxious"

Encoder: Reads input token by token → final hidden state =
"context vector"

Decoder: Takes context vector → generates output token by
token

The Bottleneck Problem

ALL information from the input must be compressed into ONE fixed-size hidden state vector. For a 1000-word input, that single vector must capture everything.

This is the exact problem that Attention (2015) and Transformers (2017) solve.

Instead of one summary vector, attention lets the decoder look at ALL encoder states.

```
In [ ]: # — Seq2Seq demonstration (simplified) —————

class SimpleSeq2Seq(nn.Module):
    """
    Minimal Seq2Seq to illustrate the concept.

    Real-world Seq2Seq (Google Translate 2016) had:
    - 8-layer LSTMs
    - Attention mechanism (we'll cover in Notebook 07)
    - Beam search decoding
    """
    def __init__(self, vocab_size, embed_dim, hidden_dim):
        super().__init__()
```

```

self.embedding = nn.Embedding(vocab_size, embed_dim)
self.encoder = nn.LSTM(embed_dim, hidden_dim, batch_first=True)
self.decoder = nn.LSTM(embed_dim, hidden_dim, batch_first=True)
self.output_proj = nn.Linear(hidden_dim, vocab_size)

def forward(self, src, tgt):
    # Encode: process input → context vector
    src_embedded = self.embedding(src)
    _, (h_n, c_n) = self.encoder(src_embedded)
    # h_n is the "context vector" – all input compressed into one vector

    # Decode: generate output from context + target
    tgt_embedded = self.embedding(tgt)
    decoder_out, _ = self.decoder(tgt_embedded, (h_n, c_n))
    logits = self.output_proj(decoder_out)

    return logits

s2s = SimpleSeq2Seq(vocab_size=100, embed_dim=32, hidden_dim=64)
src = torch.randint(0, 100, (4, 10)) # batch=4, src_len=10
tgt = torch.randint(0, 100, (4, 8)) # batch=4, tgt_len=8
out = s2s(src, tgt)
print(f'Encoder input: {src.shape}')
print(f'Decoder input: {tgt.shape}')
print(f'Output logits: {out.shape}') # (4, 8, 100) – probability over vocab

print(f'\n💡 The ENTIRE input is squeezed into one hidden state vector ({64})')
print('    For long inputs, information is inevitably lost.')
print('    Attention (Notebook 07) fixes this by allowing the decoder')
print('    to "look back" at ALL encoder positions.')

```

7. Why RNNs Lost to Transformers

Limitation	RNN	Transformer
Parallelisation	Sequential (token by token)	Fully parallel
Long-range deps	Vanishing gradients	Self-attention (direct connection)
Training speed	Slow (no parallelism)	Fast on GPUs
Scalability	Doesn't scale well	Scales to billions of params

But RNNs still matter because:

1. **Historical context:** You can't understand Transformers without understanding what they replaced
2. **Edge cases:** RNNs work well for small, sequential tasks (time series, audio)
3. **Efficiency:** Mamba, RWKV, and other recent models revive RNN ideas for efficiency
4. **Interview questions:** "Why do Transformers work better than RNNs?" is a common question

✓ Key Takeaways

1. **RNNs** process sequences step-by-step with a hidden state (memory)
2. **Vanishing gradients** prevent vanilla RNNs from learning long-range patterns
3. **LSTMs** solve this with a cell state highway and three gates (forget, input, output)
4. **GRUs** are simpler LSTMs with two gates — often perform just as well
5. **Seq2Seq** = Encoder + Decoder — but bottleneck limits performance on long inputs
6. Transformers solve ALL these limitations — that's next

Next: [07 — Attention & Transformers](#) — the architecture behind GPT, BERT, and Phi-3